Lecture 4: AI Agents & Tool Use

Building an Application Routing Agent

University of Chicago

April 9, 2026

Outline

1. Where We Are and Where We're Going
2. The Optimized Scorer: What "Solved" Looks Like
3. AI Agents
4. Readings
5. Production Considerations
6. Agentic Email Outreach
7. Your Turn: Building the Agent

Outline

1. Where We Are and Where We're Going
2. The Optimized Scorer: What "Solved" Looks Like
3. AI Agents
4. Readings
5. Production Considerations
6. Agentic Email Outreach
7. Your Turn: Building the Agent

The Story So Far

Lecture 1: Foundations

- LLMs, tokens, context windows, APIs, pricing

Lecture 2: Building a System

- Vertical slices and Crawl-Walk-Run
- Built a resume scorer: one prompt $\rightarrow$ one score

Lecture 3: Making It Good

- Decomposition, citations, few-shot examples, temperature
- Improved the scorer until gold and silver resumes separate cleanly

Result: We have a reliable process for scoring resumes.
Question: Now what do we *do* with those scores?

What We Want to Build

A hiring pipeline doesn't end at a score. After scoring, you need to:

- Schedule interviews for strong candidates
- Reject poor matches with a polite email
- Flag edge cases for human review
- Route specialists to the right department

These are **actions**, not text. Our current system can't do any of them.

Today: Give LLMs the ability to *act* — through **tools** and **agents**.

Outline

1. Where We Are and Where We're Going
2. The Optimized Scorer: What "Solved" Looks Like
3. AI Agents
4. Readings
5. Production Considerations
6. Agentic Email Outreach
7. Your Turn: Building the Agent

Recap: The Prompt That Works

After three lectures of iteration, we have a scoring prompt with:

1. **Core disqualifier checks** — missing .NET → cap at 55, no SQL → cap at 60
2. **Anchor-point scoring bands** — explicit ranges (0–30, 31–49, . . . , 93–100)
3. **Bonus/penalty system** — AWS +3, no Agile −2, frontend-only −10
4. **7 calibration examples** — from "registered nurse → 5–15" to "9 years C#/.NET + React + AWS → 93–97"

This is context engineering in action: every rule, cap, and example is steering the model toward consistent, defensible scores.

The Results

Scoring 8 resumes (2 gold, 2 silver, 4 wild) with Mistral Small:

Tier	Scores	Mean
Gold (strong match)	97, 98	97.5
Silver (weak match)	72, 72	72.0
Wild (random)	10, 15, 15, 15	13.75

- Clean separation: gold $\gg$ silver $\gg$ wild
- Total cost: $0.002 for all 8 resumes
- Uses a *cheap* model (Mistral Small) — the prompt does the heavy lifting

Takeaway: A well-engineered prompt can make even inexpensive models perform well. Now let's use these scores to *take action*.

Outline

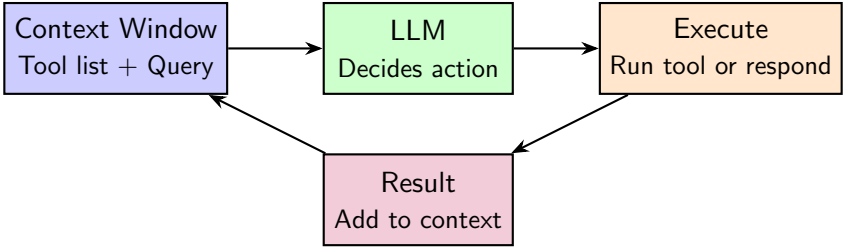
1. Where We Are and Where We're Going
2. The Optimized Scorer: What "Solved" Looks Like
3. AI Agents
4. Readings
5. Production Considerations
6. Agentic Email Outreach
7. Your Turn: Building the Agent

What is an AI Agent?

- An **AI Agent** is an LLM that can use **tools** to interact with the outside world
- While a basic LLM can only generate text based on its training data, an agent can interact with "tools" that:
  - **Database Tools**: Query databases, insert/update/delete records
  - **API Tools**: Call REST APIs, interact with web services
  - **File System Tools**: Read/write files, list directories
  - **Code Execution Tools**: Run Python code, execute shell commands
  - **Web Tools**: Search the web, scrape websites
  - ...and more

How Do They Work?

Agents are surprisingly simple:



Key idea: The LLM chooses which tool to use (or decides not to use any)

The Agent Loop

We can simplify this into "The Agent Loop"

$$\text{Observe} \rightarrow \text{Think} \rightarrow \text{Act} \rightarrow \text{Observe} \rightarrow \ldots$$

1. **Observe**: Receive input (user query, tool results, system state)
2. **Think**: Process information and decide what to do next
3. **Act**: Execute actions (call tools, generate responses)

Each turn of the loop:

1. Agent sees: current state + what's been done so far
2. Agent decides: which tool to call next (via LLM)
3. Execute tool, log result
4. Repeat until agent calls 'done' or max turns reached

Core Components of an Agent

Every agent system has four main components:

1. **System Prompt**
   - Overall system parameters
2. **Tool Registry**
   - Collection of available functions/tools with their descriptions and required parameters
   - The agent may choose from this registry
3. **Action History / Conversation Memory**
   - What tools have been called so far?
   - What were the results?
4. **User Prompt / Task**
   - The user-provided prompt

The Tool Ecosystem

Different approaches for defining and sharing tools:

- **MCP (Model Context Protocol)**
  - Standardized protocol for connecting AI assistants to external tools and data
  - Developed by Anthropic; allows tools to be shared across different AI systems
  - Example: Database connections, API integrations, file system access
- **agents.txt**
  - Markdown file describing available tools and agent capabilities
  - Simple, human-readable format for tool discovery
  - Often placed in project root for documentation
- **Skills / Plugins**
  - Pre-packaged tool collections for specific tasks
  - Examples: PDF parsing, web scraping, git operations
  - Can be installed and configured per-project

Example: Weather Query (Context)

Agent has access to a web search tool

User asks: "What's the weather tomorrow in Chicago?"

Context window contains:

- **System Prompt:**
  - `Use the tools below to answer the user's query. Return your decision as JSON: {"tool": ..., "parameters": {...}, "reasoning": "..."}`
- **Tool Registry**:
  - `web_search(query: str) -> str`: Search the web and return results
  - `done(message: str)`: Signal completion with a final message
- **User Query**: "What's the weather tomorrow in Chicago?"

CODE sends this to LLM

Example: Weather Query (Response)

LLM Response (Turn 1):

JSON Output

```
{
  "tool": "web_search",
  "parameters": {
    "query": "Chicago weather tomorrow forecast"
  },
  "reasoning": "Need to search for current weather
                forecast to answer the user's question"
}
```

CODE Response:

1. Parses this JSON
2. Executes `web_search("Chicago weather tomorrow forecast")`
3. Gets result: "Tomorrow: 45 deg F, partly cloudy, 20% chance of rain"
4. Adds result to context for next turn

Example: Weather Query (Turn 2)

Updated context window now includes:

- Original query
- Tool registry
- Previous action: Called `web_search`
- Previous result: "Tomorrow: 45 deg F, partly cloudy, 20% chance of rain"

CODE Sends this to LLM

LLM Response (Turn 2):

JSON Output

```
{
  "tool": "done",
  "parameters": {
    "message": "Tomorrow in Chicago will be 45 deg F with
                partly cloudy skies and a 20% chance of rain."
  },
  "reasoning": "I have the weather information needed
                to answer the user's question"
}
```

CODE Response

- Since the tool is "done", the code knows to stop the loop and return control to the user.

Implementing the Loop in Code

This is just a simple loop!:

Pseudo-code

```
action_history = []
for turn in range(1, max_turns):
    decision = call_llm(context, tools, action_history)     # LLM

    tool_name = decision['tool']
    params = decision['parameters']
    result = execute_tool(tool_name, params) # execute tool

    action_history.append({ 'tool': tool_name, 'result': result}) # update context

    if tool_name == 'done':     # Check if done
        break
```

Outline

1. Where We Are and Where We're Going
2. The Optimized Scorer: What "Solved" Looks Like
3. AI Agents
4. Readings
5. Production Considerations
6. Agentic Email Outreach
7. Your Turn: Building the Agent

Required Reading

Agents and Tools

- Building Effective Agents — Anthropic's guide to agent patterns and best practices

Reading Takeaway: Start Simple

Anthropic's #1 finding: The most successful agent deployments use simple, composable patterns — not fancy frameworks.

Their recommended progression:

1. Start with a single, well-engineered LLM call
2. Add retrieval and structured output
3. Only then consider multi-step agents

Sound familiar?

- Lecture 2: single prompt $\rightarrow$ score
- Lecture 3: decomposition, citations, few-shot
- Lecture 4 (today): agent loop with tools

You've been following best practice all along. Most real-world problems **don't need agents** — a good prompt is enough.

Reading Takeaway: Tool Design > Prompt Design

Anthropic spent more time optimizing tools than prompts on their coding agent.

Why? Because tools are the **interface** the model uses to act. Bad tool design → bad decisions.

What makes a good tool?

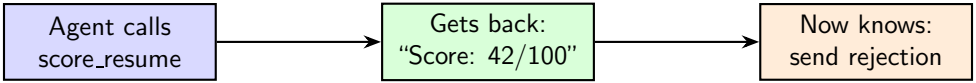
- **Clear descriptions** — would a new employee understand what this tool does?
- **Examples** in the tool description (edge cases, expected formats)
- **Obvious parameters** — don't make the model guess
- **Error-proof design** — when relative file paths caused bugs, they switched to absolute paths

Lesson: Treat tool descriptions like you treat prompts — iterate, test, improve.

Reading Takeaway: Ground Truth From the Environment

What makes agents different from a chain of prompts?

Agents get **real information back** from tools that they didn't have before.



- The agent **couldn't decide** on turn 1 — it needed the score first
- The tool result **changed what happened next**
- This is why the loop exists: each step depends on what came before

Without this feedback, you just have a checklist — not an agent.

Outline

1. Where We Are and Where We're Going
2. The Optimized Scorer: What "Solved" Looks Like
3. AI Agents
4. Readings
5. Production Considerations
6. Agentic Email Outreach
7. Your Turn: Building the Agent

Common Failure Modes

Agents can fail in predictable ways:

1. **Infinite Loops / Repeated Actions**
   - Agent calls the same tool repeatedly without progress
   - *Mitigation*: Track action history, detect cycles, set max turns
2. **Tool Hallucination**
   - Agent invents tool names that don't exist in the registry
   - *Mitigation*: Validate tool names, provide clear error messages
3. **Parameter Errors**
   - Agent provides wrong types or missing required parameters
   - *Mitigation*: Schema validation, examples in tool descriptions
4. **Premature Completion**
   - Agent calls 'done' before completing necessary actions
   - *Mitigation*: Clear instructions about required steps, validate state

Ralph Wiggum: Preventing Premature Completion

A viral tool for preventing premature completion

Ralph Wiggum Plugin

Implementation of the Ralph Wiggum technique for iterative, self-referential AI development loops in Claude Code.

What is Ralph?

Ralph is a development methodology based on continuous AI agent loops. As Geoffrey Huntley describes it: **"Ralph is a Bash loop"** - a simple `while true` that repeatedly feeds an AI agent a prompt file, allowing it to iteratively improve its work until completion.

The technique is named after Ralph Wiggum from The Simpsons, embodying the philosophy of persistent iteration despite setbacks.

Core Concept

This plugin implements Ralph using a **Stop hook** that intercepts Claude's exit attempts:

```
# You run ONCE:
/ralph-loop "Your task description" --completion-promise "DONE"

# Then Claude Code automatically:
# 1. Works on the task
# 2. Tries to exit
# 3. Stop hook blocks exit
```

Real-World and Ethical Considerations

Agents act in the world — that raises the stakes.

Engineering

- **Safety**: Max turn limits, cost caps, emergency stops
- **Observability**: Log every tool call, track costs, monitor success rates
- **Reliability**: Validate parameters, retry on transient errors, fall back to humans

Ethics

- **Bias**: Agents perpetuate training data biases — resume screening can disadvantage demographics
- **Transparency**: Decisions must be explainable to stakeholders
- **Accountability**: Who is responsible when the agent is wrong?

The common answer: human-in-the-loop

- Design explicit "flag for review" tools for high-stakes, edge-case, or low-confidence decisions
- Keep an audit trail of every action and why it was taken

Outline

1. Where We Are and Where We're Going
2. The Optimized Scorer: What "Solved" Looks Like
3. AI Agents
4. Readings
5. Production Considerations
6. Agentic Email Outreach
7. Your Turn: Building the Agent

Agentic Resume Outreach

An agentic system that scores resumes and drafts personalized emails

- Tools make **real LLM calls** — the agent genuinely needs the loop
- Agent can't draft the email until it knows the score
- Each tool creates new information that flows into the next decision

Components

1. **Tool Registry** — Python functions the agent can call
   - `score_resume` — runs the L3 optimized scorer (real LLM call)
   - `draft_outreach_email` — generates a personalized email (real LLM call)
   - `done` — signals processing is complete
2. **Agent Loop** — Multi-turn decision making
   - Observe action history
   - Decide which tool to call next
   - Execute tool and log result
   - Repeat until done
3. **Routing Rules**
   - Score $\geq 80 \rightarrow$ INTERVIEW
   - Score 40–79 $\rightarrow$ REVIEW
   - Score $< 40 \rightarrow$ REJECT

Tool Example: score_resume

A tool is a Python function that makes a real LLM call

Python Code

```
def score_resume(candidate_id):
    """Score a candidate's resume. Returns score
    (0-100) and reasoning."""
    result = structured_llm_call(
        api_key=OPENROUTER_API_KEY,
        prompt=scoring_prompt,
        context_data={"resume": resume_text},
        output_schema={"score": "integer 0-100",
                       "reasoning": "1-3 sentences"},
        model=MODEL,
    )
    return {"status": "success",
            "score": result['result']['score'],
            "message": f"Score: {score}/100"}
```

Tool Registry

- The **Tool Registry** is a Python dictionary passed to the agent
- The LLM sees what tools are available and decides which to call
- Each entry contains:
  - **function**: the callable Python function
  - **description**: what the tool does (used in the agent prompt)
  - **parameters**: expected inputs
- You define the tools — `agent_utils.py` provides the loop machinery

Example Agent Flow

Candidate: 7 years C#/.NET, React, SQL Server

Turn 1:

- Observe: No actions taken yet
- Decide: `score_resume`
- Execute: Score 88/100 — strong .NET/React/SQL match

Turn 2:

- Observe: Score is 88 → INTERVIEW
- Decide: `draft_outreach_email`(outcome=INTERVIEW)
- Execute: Personalized interview invite drafted (312 chars)

Turn 3:

- Observe: Score and email both complete
- Decide: `done`
- Execute: Processing complete

Outline

1. Where We Are and Where We're Going
2. The Optimized Scorer: What "Solved" Looks Like
3. AI Agents
4. Readings
5. Production Considerations
6. Agentic Email Outreach
7. Your Turn: Building the Agent

Today's Workflow

You will build an email outreach agent: Open the notebook:

`lecture_4_application_routing_agent.ipynb`

Define 3 tools (score, email, done)

Run the agent loop and watch it work

Read and compare the drafted emails

Submit to the leaderboard